

another step in the right direction. But we are still not at the maximum revenue rate, and we will have to take another step to get closer.

Turning the Steps into an Algorithm

You can see the pattern that is emerging. We're following these steps repeatedly:

1. Start with a `current_rate` and a `step_size`.
2. Calculate the derivative of the function you are trying to maximize at the `current_rate`.
3. Add `step_size*revenue_derivative(current_rate)` to the current rate, to get a `new current_rate`.
4. Repeat steps 2 and 3.

The only thing that's missing is a rule for when to stop, a rule that triggers when we have reached the maximum. In practice, it's quite likely that we'll be *asymptotically* approaching the maximum: getting closer and closer to it but always remaining microscopically distant. So although we may never reach the maximum, we can get close enough that we match it up to 3 or 4 or 20 decimal places. We will know when we are sufficiently close to the asymptote when the amount by which we change our rate is very small. We can specify a threshold for this in Python:

```
threshold = 0.0001
```

Our plan is to stop our process when we are changing the rate by less than this amount at each iteration of our process. It's possible that our step-taking process will never converge to the maximum we are seeking, so if we set up a loop, we'll get stuck in an infinite loop. To prepare for this possibility, we'll specify a number of "maximum iterations," and if we take a number of steps equal to this maximum, we'll simply give up and stop.

Now, we can put all these steps together ([Listing 3-1](#)).

```
threshold = 0.0001
maximum_iterations = 100000
```
